

Université Hassan II de
Casablanca
Faculté des Sciences & Techniques
Mohammedia

Licence Informatique, Réseaux & Multimédia

Collecte de Données Multimodales

Réalité Texte

Guide Complet des Mécanismes d'Acquisition du Texte

Réalisé par :
Prof. Abdellah ADIB

May 1, 2026

Contents

1	Introduction	3
1.1	Contexte et Objectifs	3
1.2	La Réalité Texte	3
1.3	Vue d'Ensemble des Mécanismes	3
2	Saisie Clavier	4
2.1	Principe Fondamental	4
2.1.1	Codes de Touche et Caractères	4
2.1.2	Protocoles de Communication	4
2.2	Impémentation avec Python	4
2.2.1	Installation des Dépendances	4
2.2.2	Lecture Simple de l'Entré Clavier	4
2.2.3	Collecteur de Texte avec Tampon	5
2.2.4	Saisie Asynchrone avec asynio	7
2.3	Bonnes Pratiques	9
3	OCR — Reconnaissance Optique de Caractères	10
3.1	Principe Fondamental	10
3.1.1	Étapes du Processus	10
3.1.2	Approches Traditionnelles vs Deep Learning	10
3.2	OCR avec Tesseract	10
3.2.1	Installation	11
3.2.2	OCR Simple sur une Image	11
3.2.3	Prétraitement pour Améliorer la Précision	11
3.2.4	Modes de Segmentation (PSM)	13
3.3	OCR avec Deep Learning : EasyOCR	14
3.3.1	Installation	14
3.3.2	Utilisation Basique	14
3.4	Comparaison des Outils OCR	15
4	Reconnaissance Vocale — Speech-to-Text	16
4.1	Introduction à la Reconnaissance Vocale	16
4.1.1	Composants Clés	16
4.1.2	Types de Reconnaissance	16
4.2	Reconnaissance avec Google Speech Recognition	16
4.2.1	Installation	16
4.2.2	Transcription depuis un Fichier Audio	16
4.2.3	Transcription depuis le Microphone	17
4.3	Reconnaissance Offline avec OpenAI Whisper	19
4.3.1	Installation	19
4.3.2	Transcription avec Whisper	19

4.3.3	Comparaison des Modèles Whisper	21
5	Codes-Barres & QR Codes	22
5.1	Introduction	22
5.1.1	Différences 1D vs 2D	22
5.2	Décodage avec Python	22
5.2.1	Installation	22
5.2.2	Décodage d'une Image	22
5.2.3	Génération de QR Codes	24
5.2.4	Scan en Temps Réel avec Caméra	25
6	Reconnaissance de l'Écriture Manuscrite	28
6.1	Introduction	28
6.2	Reconnaissance avec Google Cloud Vision	28
6.2.1	Installation	28
6.2.2	Reconnaissance Manuscrite via API	28
6.3	Reconnaissance avec un Modèle Local (Keras/TensorFlow)	30
6.4	Stylus Numérique	33
7	Conclusion et Bonnes Pratiques	36
7.1	Récapitulatif des Mécanismes	36
7.2	Bonnes Pratiques Générales	36
7.3	Arbre de Décision	36
8	Bibliographie et Ressources	38
8.1	Ouvrages de Référence	38
8.2	Documentation Officielle	38
8.3	Tutoriels Recommandés	38
8.4	Jeux de Données pour Pratiquer	39
8.5	Aspects Légaux et Éthiques	39

Chapitre 1 Introduction

1.1 Contexte et Objectifs

La collecte de données constitue la première étape fondamentale de tout projet d'intelligence artificielle et d'apprentissage automatique. Sans données de qualité, même les algorithmes les plus sophistiqués ne peuvent produire de résultats pertinents. Ce rapport se concentre sur la **Réalité Texte** de la collecte de données, c'est-à-dire l'ensemble des mécanismes permettant de transformer des informations brutes — qu'elles soient physiques, numériques ou audio — en données textuelles exploitables par un système informatique.

1.2 La Réalité Texte

La Réalité Texte répond à la question fondamentale : *“Comment transformer une source d'information en texte numérique ?”*

Cette dimension de la collecte se caractérise par :

- **Dépendance au support** : Les mécanismes varient selon la nature de la source (clavier, document scanné, parole, code-barres, écriture manuscrite).
- **Conversion de modalité** : L'accent est mis sur la transformation d'une modalité source (image, audio, signal) en modalité texte.
- **Standardisation de la sortie** : Le résultat est toujours une chaîne de caractères en format UTF-8, indépendamment du mécanisme utilisé.

1.3 Vue d'Ensemble des Mécanismes

Ce rapport couvre en détail les six principaux mécanismes d'acquisition textuelle :

1. **Saisie Clavier** : Entrée directe via périphérique de saisie
2. **OCR (Reconnaissance Optique de Caractères)** : Extraction depuis des images de texte
3. **Reconnaissance Vocale (Speech-to-Text)** : Conversion de l'audio en texte
4. **Codes-Barres & QR Codes** : Décodage de données structurées
5. **Reconnaissance Écriture Manuscrite** : Extraction depuis du texte manuscrit
6. **Stylus Numérique** : Capture en temps réel via un stylo numérique

Remarque

Chaque mécanisme fait l'objet d'une section détaillée avec ses principes fondamentaux, une implémentation Python complète, et des exemples pratiques.

Chapitre 2 Saisie Clavier

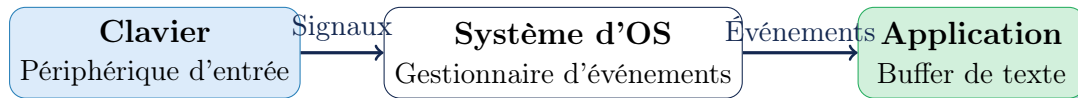


Figure 2.1: Architecture de la saisie clavier

2.1 Principe Fondamental

La saisie clavier est le mécanisme le plus simple et le plus direct pour introduire du texte dans un système informatique. Elle repose sur une architecture événementielle :

1. L'utilisateur appuie sur une touche du clavier
2. Le contrôleur du clavier génère un code de touche (*keycode*)
3. Le système d'exploitation traduit ce code en caractère Unicode
4. L'application reçoit l'événement et met à jour son tampon de texte

2.1.1 Codes de Touche et Caractères

Chaque touche physique correspond à un identifiant unique. La correspondance entre code de touche et caractère dépend de la disposition clavier (AZERTY, QWERTY, etc.) et de la localisation du système.

2.1.2 Protocoles de Communication

Les claviers modernes utilisent principalement deux protocoles :

- **USB HID** (*Human Interface Device*) : protocole standard pour les périphériques filaires et sans fil via dongles USB.
- **Bluetooth HID** : version sans fil utilisant le protocole Bluetooth Low Energy (BLE).

2.2 Impémentation avec Python

2.2.1 Installation des Dépendances

Listing 2.1: Installation des bibliotheques necessaires

```
pip install pynput pyperclip
```

2.2.2 Lecture Simple de l'Entré Clavier

Listing 2.2: Ecoute basique des evenements clavier

```
from pynput import keyboard
```

```

def on_press(key):
    """Callback appele a chaque pression de touche."""
    try:
        print(f"Caractere presse: '{key.char}')"
    except AttributeError:
        # Touche speciale (Enter, Escape, etc.)
        print(f"Touche speciale: {key}")

def on_release(key):
    """Callback appele a chaque relachement de touche."""
    if key == keyboard.Key.esc:
        print("Arret de l'ecoute.")
        return False # Arrêter le listener

# Demarrer l'ecoute
print("Appuyez sur les touches (Esc pour arreter)...")
with keyboard.Listener(on_press=on_press,
    on_release=on_release) as listener:
    listener.join()

```

2.2.3 Collecteur de Texte avec Tampon

Listing 2.3: Collecteur de texte avec gestion du tampon

```

from pynput import keyboard
import time
from dataclasses import dataclass, field
from typing import List

@dataclass
class SessionTexte:
    """Représente une session de saisie."""
    texte: str = ""
    debut: float = field(default_factory=time.time)
    fin: float = 0.0
    nb_caracteres: int = 0
    nb_mots: int = 0

    def finaliser(self):
        """Calcule les statistiques finales."""
        self.fin = time.time()
        self.nb_caracteres = len(self.texte)
        self.nb_mots = len(self.texte.split())

    def duree(self) -> float:
        """Retourne la duree en secondes."""
        fin = self.fin if self.fin > 0 else time.time()
        return fin - self.debut

```

```

class CollecteurTexte:
    """
    Collecte du texte via le clavier avec gestion
    du tampon et des sessions.
    """

    def __init__(self, delai_fin_session: float = 5.0):
        self.tampon = ""
        self.sessions: List[SessionTexte] = []
        self.session_active = None
        self.delai_fin_session = delai_fin_session
        self.derniere_pression = 0.0

    def _nouvelle_session(self):
        """Cree une nouvelle session si necessaire."""
        if self.session_active is None:
            self.session_active = SessionTexte()
            print("\n[Session demarree]")

    def _verifier_fin_session(self):
        """Verifie si la session doit etre fermee (pause
        longue)."""
        if self.session_active and time.time() -
            self.derniere_pression > self.delai_fin_session:
            self.session_active.texte = self.tampon
            self.session_active.finaliser()
            self.sessions.append(self.session_active)
            print(f"\n[Session fermee:
                {self.session_active.nb_mots} mots, "
                f"{self.session_active.duree():.1f}s]")
            self.session_active = None
            self.tampon = ""

    def on_press(self, key):
        """Gestion de la pression de touche."""
        self._verifier_fin_session()
        self._nouvelle_session()
        self.derniere_pression = time.time()

        try:
            if key.char:
                self.tampon += key.char
        except AttributeError:
            # Gestion des touches speciales
            if key == keyboard.Key.space:
                self.tampon += " "
            elif key == keyboard.Key.enter:
                self.tampon += "\n"
            elif key == keyboard.Key.backspace:
                self.tampon = self.tampon[:-1]
            elif key == keyboard.Key.tab:

```

```

        self.tampon += "\t"
    elif key == keyboard.Key.esc:
        return False # Arrêter

    return True

def on_release(self, key):
    """Gestion du relachement."""
    pass

def demarrer(self):
    """Démarrage la collecte."""
    print("Collecte active (Esc pour arrêter)...\n")
    with keyboard.Listener(
        on_press=self.on_press,
        on_release=self.on_release
    ) as listener:
        listener.join()

    # Finaliser la dernière session
    if self.session_active:
        self.session_active.texte = self.tampon
        self.session_active.finaliser()
        self.sessions.append(self.session_active)

def rapport(self):
    """Affiche un rapport de toutes les sessions."""
    print("\n" + "=" * 50)
    print("RAPPORT DE COLLECTE")
    print("=" * 50)
    for i, session in enumerate(self.sessions, 1):
        print(f"\nSession {i}:")
        print(f"  Texte: '{session.texte[:60]}...'")
        print(f"  Caractères: {session.nb_caracteres}")
        print(f"  Mots: {session.nb_mots}")
        print(f"  Durée: {session.duree():.1f}s")

# Utilisation
if __name__ == "__main__":
    collecteur = CollecteurTexte(delai_fin_session=3.0)
    collecteur.demarrer()
    collecteur.rapport()

```

2.2.4 Saisie Asynchrone avec `asyncio`

Pour les applications nécessitant une saisie non-bloquante :

Listing 2.4: Saisie clavier asynchrone

```

import asyncio
from pynput import keyboard

```



```
import queue

class SaisieAsyncClavier:
    """Saisie clavier non-bloquante via asyncio."""

    def __init__(self):
        self.file = queue.Queue()
        self.actif = True

    def _on_press(self, key):
        """Callback synchrone vers la file async."""
        if key == keyboard.Key.esc:
            self.actif = False
            self.file.put(None) # Signal d'arret
            return False
        try:
            self.file.put(key.char)
        except AttributeError:
            if key == keyboard.Key.space:
                self.file.put(" ")
            elif key == keyboard.Key.enter:
                self.file.put("\n")

    async def lire_caractere(self) -> str | None:
        """Lit un caractere de maniere asynchrone."""
        loop = asyncio.get_event_loop()
        return await loop.run_in_executor(None, self.file.get)

    def demarrer_listener(self):
        """Demarre le listener dans un thread separe."""
        import threading
        thread = threading.Thread(
            target=lambda:
                keyboard.Listener(on_press=self._on_press).start(),
            daemon=True
        )
        thread.start()

async def main():
    saisie = SaisieAsyncClavier()
    saisie.demarrer_listener()

    texte = ""
    print("Tapez (Esc pour arreter)...")

    while True:
        carac = await saisie.lire_caractere()
        if carac is None:
            break
        texte += carac
```

```
# Ici vous pouvez faire d'autres traitements async
print(f"\r> {texte}", end="", flush=True)

print(f"\n\nTexte final: '{texte}'")

asyncio.run(main())
```

2.3 Bonnes Pratiques

1. Toujours gérer les touches spéciales (Backspace, Enter, Tab)
2. Utiliser des sessions pour découper la saisie en segments
3. Implémenter un délai de détection de pause
4. Ne pas bloquer le thread principal avec l'écoute
5. Gérer l'encodage UTF-8 pour les caractères spéciaux
6. Proposer une option pour masquer la saisie (mots de passe)

Chapitre 3 OCR — Reconnaissance Optique de Caractères

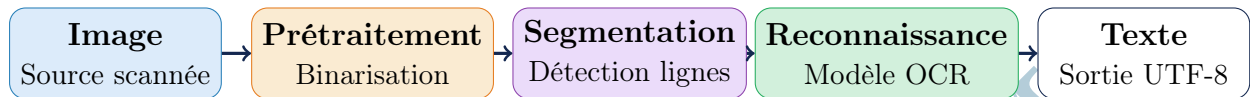


Figure 3.1: Pipeline standard d’OCR

3.1 Principe Fondamental

L’OCR (Optical Character Recognition) est le processus de conversion d’une image contenant du texte en données textuelles numériques. Cette technique est fondamentale pour la numérisation d’archives papier, la capture de reçus, ou l’extraction de texte depuis des documents scannés.

3.1.1 Étapes du Processus

Le pipeline OCR comprend typiquement quatre étapes :

1. **Prétraitement** : Conversion en niveaux de gris, binarisation (seuillage), correction de la perspective et suppression du bruit.
2. **Segmentation** : Détection des régions de texte, découpage en lignes puis en mots individuels.
3. **Reconnaissance** : Classification de chaque segment en caractères via un modèle entraîné (classique ou deep learning).
4. **Post-traitement** : Correction orthographique, assemblage du texte et structuration.

3.1.2 Approches Traditionnelles vs Deep Learning

Critère	Approche Traditionnelle	Deep Learning
Mécanisme	Templates + SVM	CNN + RNN (CRNN)
Précision	Moyenne (80–90%)	Très haute (95–99%)
Robustesse	Fragile au bruit	Très robuste
Vitesse d’entraînement	Rapide	Longue
Exemple	Tesseract (legacy)	Tesseract (LSTM), EasyOCR

3.2 OCR avec Tesseract

Tesseract est le moteur OCR open-source le plus populaire, développé initialement par HP et maintenant maintenu par Google.

3.2.1 Installation

Listing 3.1: Installation de Tesseract et du wrapper Python

```
# Linux (Ubuntu/Debian)
sudo apt-get install tesseract-ocr tesseract-ocr-fra

# Installation du wrapper Python
pip install pytesseract Pillow
```

3.2.2 OCR Simple sur une Image

Listing 3.2: Extraction de texte basique avec Tesseract

```
import pytesseract
from PIL import Image

def ocr_simple(chemin_image: str, langue: str = "fra") -> str:
    """
    Extrait le texte d'une image avec Tesseract.

    Args:
        chemin_image: Chemin vers l'image source
        langue: Code de la langue (fra, eng, etc.)

    Returns:
        Texte extrait depuis l'image
    """
    # Ouvrir l'image
    image = Image.open(chemin_image)

    # Extraction du texte
    texte = pytesseract.image_to_string(
        image,
        lang=langue,
        config="--psm 3" # Mode: bloc de texte
    )

    return texte.strip()

# Exemple d'utilisation
if __name__ == "__main__":
    texte = ocr_simple("document_scanne.jpg")
    print(f"Texte extrait:\n{texte}")
```

3.2.3 Prétraitement pour Améliorer la Précision

Listing 3.3: Pipeline de pretraitement image pour OCR

```
from PIL import Image, ImageFilter, ImageOps
```

```

import pytesseract
import numpy as np

class PretraitementOCR:
    """Ensemble de transformations pour ameliorer l'OCR."""

    @staticmethod
    def en_niveaux_de_gris(image: Image.Image) -> Image.Image:
        """Conversion en niveaux de gris."""
        return image.convert("L")

    @staticmethod
    def binarisation(image: Image.Image, seuil: int = 128) -> Image.Image:
        """Binarisation avec un seuil fixe."""
        gris = image.convert("L")
        return gris.point(lambda x: 255 if x > seuil else 0)

    @staticmethod
    def binarisation_adaptive(image: Image.Image) -> Image.Image:
        """Binarisation adaptive (meilleure pour les eclairages variables)."""
        import cv2
        import numpy as np

        gris = np.array(image.convert("L"))
        binaire = cv2.adaptiveThreshold(
            gris, 255,
            cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
            cv2.THRESH_BINARY, 11, 2
        )
        return Image.fromarray(binaire)

    @staticmethod
    def suppression_bruit(image: Image.Image) -> Image.Image:
        """Filtre median pour supprimer le bruit."""
        return image.filter(ImageFilter.MedianFilter(size=3))

    @staticmethod
    def augmentation_contraste(image: Image.Image) -> Image.Image:
        """Egalisation de l'histogramme pour ameliorer le contraste."""
        return ImageOps.equalize(image.convert("L"))

    @staticmethod
    def redimensionner(image: Image.Image, facteur: float = 2.0) -> Image.Image:
        """Agrandit l'image pour ameliorer la lisibilite."""
        nouveau_w = int(image.width * facteur)

```

```

nouveau_h = int(image.height * facteur)
return image.resize((nouveau_w, nouveau_h),
                    Image.LANCZOS)

def pipeline_ocr_ameliore(chemin_image: str) -> dict:
    """
    Pipeline OCR complet avec pretraitement et comparaison.
    """
    image_originale = Image.open(chemin_image)
    pre = PretraitementOCR()

    # Pipeline recommandé
    image_traitee = pre.en_niveaux_de_gris(image_originale)
    image_traitee = pre.suppression_bruit(image_traitee)
    image_traitee = pre.redimensionner(image_traitee, 2.0)
    image_traitee = pre.binarisation(image_traitee, seuil=140)

    # OCR sur les deux versions
    texte_original =
        pytesseract.image_to_string(image_originale, lang="fra")
    texte_traite = pytesseract.image_to_string(image_traitee,
        lang="fra")

    return {
        "original": texte_original.strip(),
        "traite": texte_traite.strip(),
        "amelioration": len(texte_traite) > len(texte_original)
    }

# Utilisation
resultats = pipeline_ocr_ameliore("scan_document.jpg")
print(f"Original: {resultats['original'][:100]}...")
print(f"Traite: {resultats['traite'][:100]}...")

```

3.2.4 Modes de Segmentation (PSM)

Tesseract offre plusieurs modes de segmentation de page :

PSM	Mode	Cas d'utilisation
0	OSD uniquement	Détection orientation
3	Bloc de texte (par défaut)	Documents standards
4	Un bloc de texte (une colonne)	Colonnes denses
6	Un bloc de texte uniforme	Blocs homogènes
7	Une ligne de texte	Lignes isolées
8	Un mot	Mots isolés
11	Sparse texte	Texte épars sur fond complexe
12	Sparse texte avec OSD	Texte épars + orientation

3.3 OCR avec Deep Learning : EasyOCR

EasyOCR utilise des modèles de deep learning (CRAFT + CRNN) pour une reconnaissance plus robuste, en particulier sur les images bruitées.

3.3.1 Installation

Listing 3.4: Installation de EasyOCR

```
pip install easyocr
```

3.3.2 Utilisation Basique

Listing 3.5: Reconnaissance OCR avec EasyOCR

```
import easyocr
import json

def ocr_easyocr(chemin_image: str, langues: list = None) -> dict:
    """
    OCR avec EasyOCR (deep learning).

    Args:
        chemin_image: Chemin vers l'image
        langues: Liste des langues a supporter

    Returns:
        Dictionnaire avec texte et scores de confiance
    """
    if langues is None:
        langues = ["fr", "en"]

    # Initialiser le lecteur (telecharge les modeles la 1ere fois)
    reader = easyocr.Reader(langues, gpu=True)

    # Reconnaissance
    resultats = reader.readtext(chemin_image)

    # Formater les resultats
    texte_extrait = []
    for (bbox, texte, confiance) in resultats:
        texte_extrait.append({
            "texte": texte,
            "confiance": round(confiance, 3),
            "position": {
                "x_min": int(bbox[0][0]),
                "y_min": int(bbox[0][1]),
                "x_max": int(bbox[2][0]),
                "y_max": int(bbox[2][1])
            }
        })
```

```

    }
    })

    # Trier par position verticale (haut vers bas)
    texte_extrait.sort(key=lambda x: x["position"]["y_min"])

    # Assemblage du texte complet
    texte_complet = " ".join([item["texte"] for item in
                               texte_extrait])

    return {
        "texte_complet": texte_complet,
        "segments": texte_extrait,
        "nb_segments": len(texte_extrait)
    }

# Utilisation
if __name__ == "__main__":
    resultats = ocr_easyocr("photo_texte.jpg")
    print(f"Texte: {resultats['texte_complet']}")
    print(f"\nDetails ({resultats['nb_segments']} segments):")
    for seg in resultats["segments"]:
        print(f"    '{seg['texte']}' (confiance:
              {seg['confiance']})")

```

3.4 Comparaison des Outils OCR

Critère	Tesseract	EasyOCR	PaddleOCR	Google Vision
Type	Open Source	Open Source	Open Source	API Cloud
Précision	Moyenne	Haute	Très haute	Très haute
Vitesse	Rapide	Moyenne	Rapide	Moyenne
Langues	100+	80+	90+	40+
GPU requis	Non	Optionnel	Optionnel	Non
Coût	Gratuit	Gratuit	Gratuit	Payant

Conseil

Pour un démarrage rapide, utilisez Tesseract avec prétraitement. Pour une précision maximale sur des images complexes, optez pour PaddleOCR ou EasyOCR.

Chapitre 4 Reconnaissance Vocale — Speech-to-Text

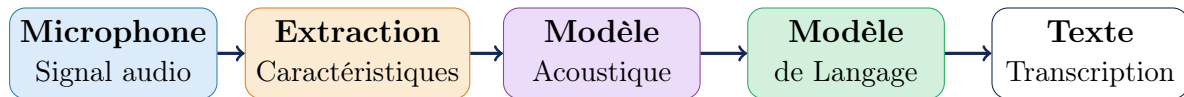


Figure 4.1: Pipeline de reconnaissance vocale

4.1 Introduction à la Reconnaissance Vocale

La reconnaissance vocale (Speech-to-Text ou STT) est la technologie permettant de convertir un signal audio contenant de la parole en texte écrit. Elle constitue un mécanisme crucial pour l'accessibilité, la dictée et l'interface vocale.

4.1.1 Composants Clés

Un système STT moderne repose sur :

- **Modèle acoustique** : Convertit les caractéristiques audio (MFCC) en séquences de phonèmes ou de tokens.
- **Modèle de langage** : Recompose des mots et phrases cohérents à partir des séquences produites par le modèle acoustique.
- **Décodeur** : Combine les deux modèles pour produire la transcription optimale.

4.1.2 Types de Reconnaissance

- **Offline (local)** : Le modèle tourne localement sur l'appareil (Whisper, Vosk).
- **Online (cloud)** : L'audio est envoyé à un serveur distant (Google, Microsoft).
- **Temps réel** : Transcription en cours pendant la parole (streaming).

4.2 Reconnaissance avec Google Speech Recognition

4.2.1 Installation

Listing 4.1: Installation des dépendances

```
pip install SpeechRecognition pyaudio
```

4.2.2 Transcription depuis un Fichier Audio

Listing 4.2: Transcription d'un fichier audio avec Google STT

```
import speech_recognition as sr
from pathlib import Path
```

```
def transcrire_fichier(chemin_audio: str, langue: str =
    "fr-FR") -> dict:
    """
    Transcrit un fichier audio avec l'API Google.

    Args:
        chemin_audio: Chemin vers le fichier audio (.wav, .flac)
        langue: Code de la langue (fr-FR, en-US, etc.)

    Returns:
        Dictionnaire avec la transcription et la confiance
    """
    recognizer = sr.Recognizer()

    # Verifier que le fichier existe
    if not Path(chemin_audio).exists():
        raise FileNotFoundError(f"Fichier non trouve:
            {chemin_audio}")

    # Charger l'audio
    with sr.AudioFile(chemin_audio) as source:
        audio_data = recognizer.record(source)

    try:
        # Transcription via Google
        texte = recognizer.recognize_google(
            audio_data,
            language=langue
        )
        return {
            "texte": texte,
            "statut": "succes",
            "langue": langue
        }
    except sr.UnknownValueError:
        return {"texte": "", "statut": "audio non compris",
            "langue": langue}
    except sr.RequestError as e:
        return {"texte": "", "statut": f"erreur API: {e}",
            "langue": langue}

# Utilisation
resultat = transcrire_fichier("enregistrement.wav", "fr-FR")
print(f"Transcription: {resultat['texte']}")
print(f"Statut: {resultat['statut']}")
```

4.2.3 Transcription depuis le Microphone

Listing 4.3: Capture et transcription en temps réel depuis le microphone

```

import speech_recognition as sr
import time

class TranscripteurMicrophone:
    """Transcription vocale depuis le microphone."""

    def __init__(self, langue: str = "fr-FR", delai_silence:
float = 2.0):
        self.recognizer = sr.Recognizer()
        self.langue = langue
        self.delai_silence = delai_silence

    def calibrer(self, duree: float = 2.0):
        """Calibre le microphone pour le bruit ambiant."""
        print(f"Calibration du microphone ({duree}s)...")
        with sr.Microphone() as source:
            self.recognizer.adjust_for_ambient_noise(source,
duration=duree)
        print("Calibration terminee.")

    def transcrire_une_fois(self) -> str | None:
        """Enregistre et transcrit une phrase."""
        print("Parlez... (silence pour terminer)")
        with sr.Microphone() as source:
            try:
                audio = self.recognizer.listen(
                    source,
                    timeout=10, # Max 10s d'attente avant
                             parole
                    phrase_time_limit=30 # Max 30s de parole
                )
                texte = self.recognizer.recognize_google(
                    audio, language=self.langue
                )
                return texte
            except sr.WaitTimeoutError:
                print("Aucune parole detectee.")
                return None
            except sr.UnknownValueError:
                print("Audio non compris.")
                return None

    def transcription_continue(self, max_phrases: int = 10):
        """Transcription continue en boucle."""
        self.calibrer()
        transcriptions = []

        for i in range(max_phrases):
            print(f"\n--- Phrase {i+1}/{max_phrases} ---")
            texte = self.transcrire_une_fois()

```

```

        if texte:
            transcriptions.append(texte)
            print(f"Transcription: '{texte}'")

    return transcriptions

# Utilisation
if __name__ == "__main__":
    transcripteur = TranscripteurMicrophone(langue="fr-FR")
    resultats =
        transcripteur.transcription_continue(max_phrases=5)
    print(f"\n== Resume ==")
    print(f"Phrases transcrites: {len(resultats)}")
    for i, phrase in enumerate(resultats, 1):
        print(f" {i}. {phrase}")

```

4.3 Reconnaissance Offline avec OpenAI Whisper

Whisper est un modèle de reconnaissance vocale open-source développé par OpenAI, capable de transcrire de l'audio dans plus de 99 langues sans nécessiter de connexion Internet.

4.3.1 Installation

Listing 4.4: Installation de Whisper

```

pip install openai-whisper
# Pour la version optimisée GPU :
pip install faster-whisper

```

4.3.2 Transcription avec Whisper

Listing 4.5: Transcription complète avec Whisper

```

import whisper
from pathlib import Path
import time

def transcrire_whisper(
    chemin_audio: str,
    modele: str = "small",
    langue: str = None
) -> dict:
    """
    Transcription avec OpenAI Whisper (offline).

    Args:
        chemin_audio: Chemin vers le fichier audio
    """

```

```

    modele: Taille du modele (tiny, base, small, medium,
        large)
    langue: Langue forcee (None = detection automatique)

Returns:
    Dictionnaire avec transcription et metadonnees
"""
if not Path(chemin_audio).exists():
    raise FileNotFoundError(f"Fichier non trouve:
        {chemin_audio}")

# Charger le modele
print(f"Chargement du modele '{modele}'...")
model = whisper.load_model(modele)

# Transcription
debut = time.time()
options = {"language": langue} if langue else {}
resultat = model.transcribe(chemin_audio, **options)
duree = time.time() - debut

return {
    "texte": resultat["text"].strip(),
    "langue_detectee": resultat["language"],
    "segments": [
        {
            "debut": seg["start"],
            "fin": seg["end"],
            "texte": seg["text"].strip()
        }
        for seg in resultat["segments"]
    ],
    "duree_traitement": round(duree, 2)
}

# Utilisation
if __name__ == "__main__":
    resultat = transcrire_whisper(
        "discours.mp3",
        modele="small",
        langue="fr"
    )
    print(f"Texte: {resultat['texte']}")
    print(f"Langue detectee: {resultat['langue_detectee']}")
    print(f"Duree traitement: {resultat['duree_traitement']}s")
    print(f"\nSegments temporels:")
    for seg in resultat["segments"]:
        print(f"    [{seg['debut']:.1f}s - {seg['fin']:.1f}s]
            {seg['texte']}")

```

4.3.3 Comparaison des Modèles Whisper

Modèle	Paramètres	Vitesse	Précision	VRAM
tiny	39M	Très rapide	Moyenne	1 Go
base	74M	Rapide	Moyenne+	1 Go
small	244M	Moyenne	Bonne	2 Go
medium	769M	Lente	Très bonne	5 Go
large	1550M	Très lente	Excellente	10 Go

Attention

Le modèle **small** représente le meilleur compromis vitesse/précision pour le français. Les modèles **medium** et **large** sont recommandés uniquement si vous disposez d'un GPU puissant.

Chapitre 5 Codes-Barres & QR Codes

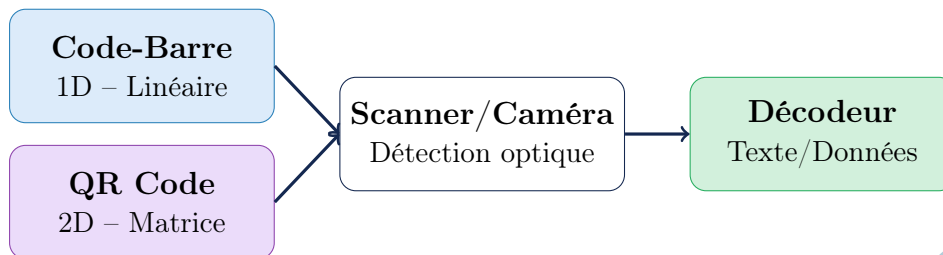


Figure 5.1: Comparaison codes-barres 1D et QR codes 2D

5.1 Introduction

Les codes-barres et QR codes sont des mécanismes d'acquisition de données encodées dans un format graphique. Ils permettent un accès rapide et fiable à des informations structurées (identifiants, URLs, données produits).

5.1.1 Différences 1D vs 2D

Caractéristique	Code-Barre 1D	QR Code 2D
Capacité	20-30 caractères	Jusqu'à 2953 octets
Direction scan	Unidirectionnel	Omnidirectionnel
Correction d'erreurs	Non	Oui (jusqu'à 30%)
Format de données	Numérique/texte	Texte, URL, binaire
Exemples	EAN-13, Code 128	QR Code, Aztec

5.2 Décodage avec Python

5.2.1 Installation

Listing 5.1: Installation des bibliothèques de décodage

```
pip install pyzbar Pillow opencv-python
# Sur Linux, installer aussi :
sudo apt-get install libzbar0
```

5.2.2 Décodage d'une Image

Listing 5.2: Décodage de codes-barres et QR codes depuis une image

```
from pyzbar.pyzbar import decode
from PIL import Image
from dataclasses import dataclass
```

```

from typing import List
import json

@dataclass
class CodeDecode:
    """Représente un code decode."""
    type_code: str          # QR_CODE, CODE_128, EAN_13, etc.
    donnees: str            # Contenu decode
    position: dict          # Coordonnees dans l'image

    def to_dict(self) -> dict:
        return {
            "type": self.type_code,
            "donnees": self.donnees,
            "position": self.position
        }

def decoder_image(chemin_image: str) -> List[CodeDecode]:
    """
    Decode tous les codes-barres et QR codes dans une image.

    Args:
        chemin_image: Chemin vers l'image a decoder

    Returns:
        Liste des codes decodes avec leurs donnees
    """
    image = Image.open(chemin_image)

    # Decodage avec pyzbar
    codes = decode(image)

    resultats = []
    for code in codes:
        # Extraire les coordonnees
        points = code.polygon
        position = {
            "x_min": min(p.x for p in points),
            "y_min": min(p.y for p in points),
            "x_max": max(p.x for p in points),
            "y_max": max(p.y for p in points)
        }

        resultats.append(CodeDecode(
            type_code=code.type,
            donnees=code.data.decode("utf-8"),
            position=position
        ))

    return resultats

```



```
# Utilisation
if __name__ == "__main__":
    codes = decoder_image("produits_rayon.jpg")

    print(f"Codes trouves: {len(codes)}")
    for i, code in enumerate(codes, 1):
        print(f"\n Code {i}:")
        print(f"     Type: {code.type_code}")
        print(f"     Donnees: {code.donnees}")
        print(f"     Position: {code.position}")
```

5.2.3 Génération de QR Codes

Listing 5.3: Generation de QR codes avec personnalisation

```
import qrcode
from PIL import Image
from pathlib import Path

def generer_qr(
    donnees: str,
    chemin_sortie: str,
    couleur_avant: str = "black",
    couleur_arriere: str = "white",
    taille_case: int = 10,
    marge: int = 4
) -> str:
    """
    Genere un QR code personnalise.

    Args:
        donnees: Donnees a encoder (URL, texte, etc.)
        chemin_sortie: Chemin de sauvegarde
        couleur_avant: Couleur des modules (noir par default)
        couleur_arriere: Couleur du fond
        taille_case: Taille de chaque module en pixels
        marge: Marge en nombre de modules

    Returns:
        Chemin du fichier QR code genere
    """
    qr = qrcode.QRCode(
        version=1, # 1 a 40 (taille)
        error_correction=qrcode.constants.ERROR_CORRECT_H, #
            30% correction
        box_size=taille_case,
        border=marge,
    )
```

```
qr.add_data(donnees)
qr.make(fit=True)

# Créer l'image
img = qr.make_image(
    fill_color=couleur_avant,
    back_color=couleur_arriere
)

# Sauvegarder
img.save(chemin_sortie)
print(f"QR Code sauvegarde: {chemin_sortie}")

return chemin_sortie

# Exemples
if __name__ == "__main__":
    # QR Code simple avec URL
    generer_qr(
        "https://www.exemple.com/produit/12345",
        "qr_url.png"
    )

    # QR Code avec donnees JSON
    import json
    donnees_produit = {
        "ref": "PROD-2024-001",
        "nom": "Smartphone X",
        "prix": 599.99
    }
    generer_qr(
        json.dumps(donnees_produit, ensure_ascii=False),
        "qr_produit.png",
        couleur_avant="#1a5276",
        couleur_arriere="#eaf2f8"
    )
```

5.2.4 Scan en Temps Réel avec Caméra

Listing 5.4: Detection et decodage en temps reel via webcam

```
import cv2
from pyzbar.pyzbar import decode
import time

def scan_temps_reel(duree_max: int = 30):
    """
    Scan de codes-barres en temps reel via la webcam.

    Args:
```

```
    duree_max: Duree maximale en secondes
"""
# Ouvrir la camera
cap = cv2.VideoCapture(0)
if not cap.isOpened():
    print("Erreur: impossible d'ouvrir la camera.")
    return []

codes_detectes = {}
debut = time.time()

print("Scan en cours... (Q pour quitter)")

while time.time() - debut < duree_max:
    ret, frame = cap.read()
    if not ret:
        break

    # Decoder les codes dans la frame
    codes = decode(frame)

    for code in codes:
        donnees = code.data.decode("utf-8")

        # Dessiner un rectangle autour du code
        points = [p for p in code.polygon]
        cv2.polylines(frame, [cv2.UMat(points)], True, (0,
            255, 0), 3)

        # Afficher les donnees
        cv2.putText(frame, donnees[:30], (code.rect.x,
            code.rect.y - 10),
            cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255,
            0), 2)

        # Enregistrer si nouveau
        if donnees not in codes_detectes:
            codes_detectes[donnees] = {
                "type": code.type,
                "timestamp": time.time()
            }
            print(f"[{code.type}] {donnees}")

    # Afficher la frame
    cv2.imshow("Scanner", frame)

    # Verifier la touche Q
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

cap.release()
```

```
cv2.destroyAllWindows()

return codes_detectes

# Utilisation
codes = scan_temps_reel(duree_max=60)
print(f"\nTotal codes scannes: {len(codes)}")
```

Licence IRM

Chapitre 6 Reconnaissance de l'Écriture Manuscrite

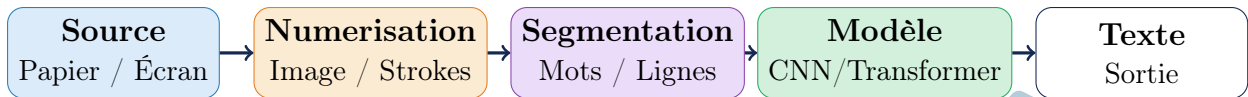


Figure 6.1: Pipeline de reconnaissance écriture manuscrite

6.1 Introduction

La reconnaissance de l'écriture manuscrite (Handwriting Recognition ou HWR) est un défi plus complexe que l'OCR standard car l'écriture manuscrite varie considérablement entre les individus. Elle se décline en deux catégories :

- **Hors ligne (offline)** : L'image du texte manuscrite est analysée après coup (documents scannés, photos).
- **En ligne (online)** : Les traits sont capturés en temps réel pendant l'écriture (tablette, stylus numérique).

6.2 Reconnaissance avec Google Cloud Vision

6.2.1 Installation

Listing 6.1: Installation du SDK Google Cloud Vision

```
pip install google-cloud-vision
```

6.2.2 Reconnaissance Manuscrite via API

Listing 6.2: Reconnaissance écriture manuscrite avec Google Cloud Vision

```
from google.cloud import vision
from pathlib import Path
import base64

class ReconnaissseusManuscrite:
    """Client pour la reconnaissance ecriture manuscrite."""

    def __init__(self, chemin_cle_api: str = None):
        """
        Initialise le client Vision API.

        Args:
```

```
        chemin_cle_api: Chemin vers le fichier JSON de cle
        de service
    """
    if chemin_cle_api:
        import os
        os.environ["GOOGLE_APPLICATION_CREDENTIALS"] =
            chemin_cle_api
    self.client = vision.ImageAnnotatorClient()

def reconnaitre_fichier(self, chemin_image: str) -> dict:
    """
    Reconnait le texte manuscrit dans une image.

    Args:
        chemin_image: Chemin vers l'image

    Returns:
        Dictionnaire avec texte et details par bloc
    """
    # Lire l'image
    with open(chemin_image, "rb") as f:
        contenu = f.read()

    image = vision.Image(content=contenu)

    # Appel a l'API avec detection de texte amelioree
    response =
        self.client.document_text_detection(image=image)
    document = response.text_annotations

    if not document:
        return {"texte": "", "blocs": [], "confiance": 0.0}

    # Texte complet
    texte_complet = document[0].description

    # Details par bloc
    blocs = []
    if response.full_text_annotation:
        for page in response.full_text_annotation.pages:
            for bloc in page.blocks:
                texte_bloc = ""
                for para in bloc.paragraphs:
                    for mot in para.words:
                        texte_bloc += "".join(
                            [s.text for s in mot.symbols]
                        ) + " "
                blocs.append({
                    "texte": texte_bloc.strip(),
                    "confiance": bloc.confidence
                })
```

```

    confiance_moy = sum(b["confiance"] for b in blocs) /
        max(len(blocs), 1)
    return {
        "texte": texte_complet,
        "blocs": blocs,
        "confiance": round(confiance_moy, 3)
    }

# Utilisation
if __name__ == "__main__":
    rec = ReconnaissseusManuscrite("cle_service.json")
    resultat = rec.reconnaitre_fichier("lettre_manuscript.jpg")

    print(f"Texte extrait:\n{resultat['texte']}")
    print(f"\nConfiance moyenne: {resultat['confiance']}")
    print(f"\nBlocs detectes:")
    for i, bloc in enumerate(resultat["blocs"], 1):
        print(f"  Bloc {i} (conf: {bloc['confiance']:.2f}):
            {bloc['texte'][:50]}...")

```

6.3 Reconnaissance avec un Modèle Local (Keras/TensorFlow)

Pour une solution 100% locale sans dépendance cloud :

Listing 6.3: Reconnaissance manuscrite avec modele CNN local

```

import numpy as np
from PIL import Image, ImageOps

class ReconnaisseurLocal:
    """
    Reconnaisseur de caracteres manuscrit avec CNN.
    Utilise un modele preentaine sur MNIST/IAM.
    """

    def __init__(self, chemin_modele: str):
        """
        Charge un modele Keras preentaine.

        Args:
            chemin_modele: Chemin vers le fichier .h5 ou
                           SavedModel
        """
        import tensorflow as tf
        self.modele = tf.keras.models.load_model(chemin_modele)
        # Taille d'entree attendue par le modele
        self.input_shape = self.modele.input_shape[1:] # ex:
            (28, 28, 1)

```

```
def pretraiter_image(self, image: Image.Image) ->
    np.ndarray:
    """
    Pretraite une image pour l'entree du modele.

    Args:
        image: Image PIL du caractere

    Returns:
        Tableau numpy normalise
    """
    # Convertir en niveaux de gris
    image = image.convert("L")

    # Inverser (fond blanc -> noir pour MNIST)
    image = ImageOps.invert(image)

    # Redimensionner
    h, w = self.input_shape[:2]
    image = image.resize((w, h), Image.LANCZOS)

    # Convertir en numpy et normaliser
    arr = np.array(image, dtype=np.float32) / 255.0

    # Ajouter les dimensions necessaires
    arr = arr.reshape(1, h, w, 1)

    return arr

def predire(self, image: Image.Image) -> dict:
    """
    Predit le caractere dans une image.

    Args:
        image: Image PIL contenant un caractere

    Returns:
        Dictionnaire avec caractere predit et confiance
    """
    # Pretraiter
    arr = self.pretraiter_image(image)

    # Prediction
    predictions = self.modele.predict(arr, verbose=0)[0]

    # Meilleure prediction
    indice = np.argmax(predictions)
    confiance = float(predictions[indice])

    # Convertir indice en caractere
```



```

caractere = self._indice_vers_caractere(indice)

return {
    "caractere": caractere,
    "confiance": round(confiance, 4),
    "top3": [
        {
            "caractere": self._indice_vers_caractere(i),
            "confiance": round(float(predictions[i]), 4)
        }
        for i in np.argsort(predictions)[-3:][::-1]
    ]
}

def _indice_vers_caractere(self, indice: int) -> str:
    """Convertit un indice de classe en caractere."""
    # Pour MNIST: 0-9
    if indice < 10:
        return str(indice)
    # Pour des modeles plus grands incluant les lettres
    return chr(ord("A") + indice - 10)

def reconnaitre_mot(self, image: Image.Image) -> str:
    """
    Segmente et reconnait chaque caractere d'un mot.

    Args:
        image: Image PIL contenant un mot manuscrit

    Returns:
        Mot reconnu
    """
    # Conversion en niveaux de gris et binarisation
    gris = np.array(image.convert("L"))
    binaire = gris < 128 # Seuillage simple

    # Projection verticale pour segmenter les caracteres
    projection = binaire.sum(axis=0)

    # Trouver les segments de caracteres
    segments = []
    dans_caractere = False
    debut = 0

    for i, val in enumerate(projection):
        if val > 0 and not dans_caractere:
            debut = i
            dans_caractere = True
        elif val == 0 and dans_caractere:
            segments.append((debut, i))
            dans_caractere = False

```

```

    if dans_caractere:
        segments.append((debut, len(projection)))

    # Reconnaître chaque segment
    mot = ""
    for debut, fin in segments:
        if fin - debut < 3: # Ignorer les segments trop
            petits
            continue
        segment_img = image.crop((debut, 0, fin,
            image.height))
        resultat = self.predire(segment_img)
        mot += resultat["caractere"]
    return mot

# Utilisation
if __name__ == "__main__":
    rec = ReconnaisseurLocal("modele_mnist.h5")

    # Reconnaître un seul caractere
    img = Image.open("caractere.png")
    resultat = rec.predire(img)
    print(f"Caractere: '{resultat['caractere']}' "
        f"(confiance: {resultat['confiance']})")
    print(f"Top 3: {resultat['top3']}")

```

6.4 Stylus Numérique

Le stylus numérique permet la reconnaissance en temps réel pendant l'écriture. Il capture non seulement l'image finale mais aussi les **traits** (strokes) : la séquence de points avec leur timing.

Listing 6.4: Capture de traits manuscrit via stylus (API exemple)

```

import time
from dataclasses import dataclass, field
from typing import List

@dataclass
class Point:
    """Un point sur l'ecran."""
    x: float
    y: float
    timestamp: float = field(default_factory=time.time)
    pression: float = 0.5 # 0.0 a 1.0

@dataclass
class Trait:
    """Un trait (stroke) continu."""
    points: List[Point] = field(default_factory=list)

```

```

debut: float = field(default_factory=time.time)
fin: float = 0.0

def ajouter_point(self, x: float, y: float, pression: float
    = 0.5):
    self.points.append(Point(x, y, time.time(), pression))

def finaliser(self):
    self.fin = time.time()

@property
def longueur(self) -> float:
    """Calcule la longueur approximee du trait."""
    if len(self.points) < 2:
        return 0.0
    total = 0.0
    for i in range(1, len(self.points)):
        dx = self.points[i].x - self.points[i-1].x
        dy = self.points[i].y - self.points[i-1].y
        total += (dx**2 + dy**2) ** 0.5
    return total

class CollecteurTraits:
    """Collecte les traits d'écriture du stylos."""

    def __init__(self):
        self.traits: List[Trait] = []
        self.trait_actif: Trait | None = None

    def debut_trait(self, x: float, y: float, pression: float =
        0.5):
        """Debut d'un nouveau trait."""
        self.trait_actif = Trait()
        self.trait_actif.ajouter_point(x, y, pression)

    def point_trait(self, x: float, y: float, pression: float =
        0.5):
        """Ajout d'un point au trait en cours."""
        if self.trait_actif:
            self.trait_actif.ajouter_point(x, y, pression)

    def fin_trait(self):
        """Fin du trait en cours."""
        if self.trait_actif:
            self.trait_actif.finaliser()
            self.traits.append(self.trait_actif)
            self.trait_actif = None

    def exporter(self) -> dict:
        """Exporte tous les traits en dictionnaire."""
        return {

```

```
"nb_traits": len(self.traits),
"traits": [
    {
        "nb_points": len(trait.points),
        "longueur": round(trait.longueur, 2),
        "points": [
            {"x": p.x, "y": p.y, "t": p.timestamp,
             "p": p.pression}
            for p in trait.points
        ]
    }
    for trait in self.traits
]
```

Chapitre 7 Conclusion et Bonnes Pratiques

7.1 Récapitulatif des Mécanismes

Mécanisme	Cas d'usage	Outils Python
Saisie Clavier	Entrée directe utilisateur	pynput, pyperclip
OCR	Documents scannés, images texte	Tesseract, EasyOCR, PaddleOCR
Speech-to-Text	Audio, dictée vocale	Whisper, SpeechRecognition
Codes-Barres / QR	Identifiants, URLs	pyzbar, qrcode
Écriture Manuscrite	Notes, formulaires	Google Vision, modèles CNN
Stylus Numérique	Entrée en temps réel	API constructeur, Wacom SDK

7.2 Bonnes Pratiques Générales

1. **Choisir le bon mécanisme** : Adaptez votre choix à la source de données (image, audio, clavier, code-barre).
2. **Valider les données** : Vérifiez toujours la qualité du texte extrait avec des scores de confiance.
3. **Gérer les erreurs** : Implémentez des retry et des fallback pour les API cloud.
4. **Optimiser le prétraitement** : La qualité de l'entrée (image, audio) a un impact direct sur la précision de la sortie.
5. **Respecter la vie privée** : Attention aux réglementations (RGPD) lors du traitement de données personnelles.
6. **Encoder en UTF-8** : Assurez-vous que toutes les sorties sont en UTF-8 pour une compatibilité universelle.
7. **Tester avec diversité** : Utilisez des données de test variées (langues, accents, qualités d'image).
8. **Surveiller les performances** : Loggez les temps de traitement et les taux d'erreur en production.

7.3 Arbre de Décision

Pour choisir le bon mécanisme d'acquisition textuelle :

1. **La source est un clavier ?**
 - Oui → Saisie Clavier (pynput)
 - Non → Continuer
2. **La source est un document/image ?**
 - Texte imprimé → OCR (Tesseract / EasyOCR)
 - Texte manuscrit → Reconnaissance HWR (Google Vision / CNN)
 - Code-barre ou QR → Décodage (pyzbar)

- Non → Continuer
3. **La source est audio ?**
- Offline (fichier) → Whisper
 - En temps réel → Google STT / Azure Speech
 - Non → Continuer
4. **La source est un stylos ?**
- Oui → API stylus + reconnaissance online

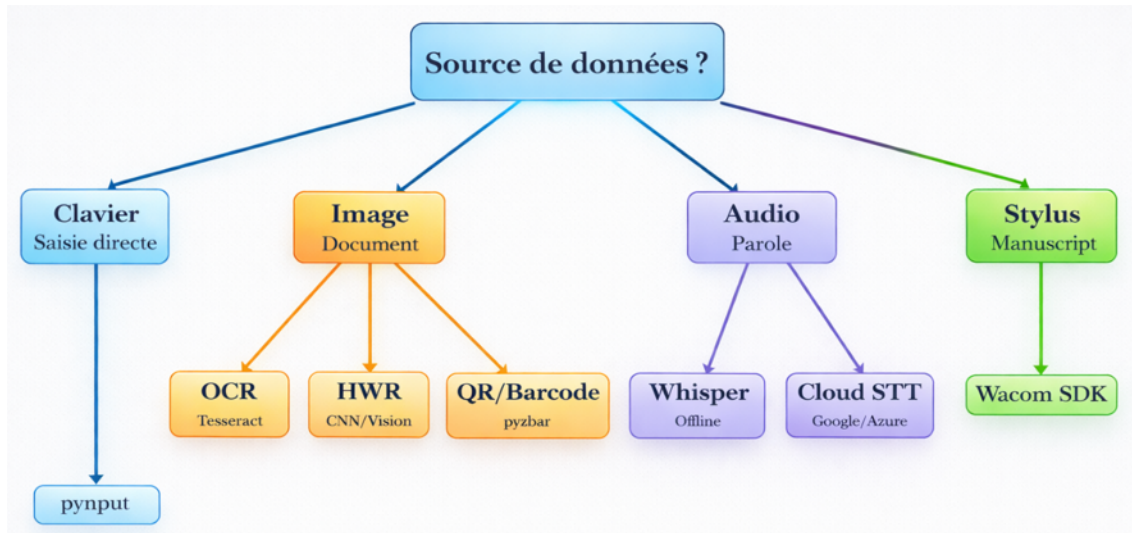


Figure 7.1: Arbre de décision pour choisir un mécanisme d'acquisition textuelle

Chapitre 8 Bibliographie et Ressources

8.1 Ouvrages de Référence

Pour approfondir vos connaissances, voici les ouvrages recommandés :

- Mitchell, R. (2018). *Web Scraping with Python: Collecting More Data from the Modern Web*, 2nd ed. O'Reilly Media. — La référence pour la collecte de données avec Python.
- Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 2nd ed. O'Reilly Media. — Excellent pour comprendre les modèles CNN utilisés en OCR et reconnaissance.
- Graves, A. (2013). *Generating Sequences With Recurrent Neural Networks*. arXiv. — Fondation théorique pour la reconnaissance séquentielle (manuscript).
- Shi, B., Bui, X., Zhu, X. & Bel, S. (2017). *An End-to-End Trainable Neural Network for Image-based Sequence Recognition*. IEEE TPAMI. — Article de référence sur CRNN pour la reconnaissance de texte.
- Radford, A. et al. (2022). *Robust Speech Recognition via Large-Scale Weak Supervision*. OpenAI. — Article décrivant le modèle Whisper.
- Irvin, J. et al. (2019). *CheXpert: A Large Chest Radiograph Dataset with Uncertainty Labels and Expert Comparison*. AAAI. — Exemple de collecte de données multimodales en contexte médical.

8.2 Documentation Officielle

Outil	URL	Description
Tesseract	tesseract-ocr.github.io	Moteur OCR open source
EasyOCR	github.com/JaidedAI/EasyOCR	OCR avec deep learning
PaddleOCR	github.com/PaddlePaddle/PaddleOCR	OCR haute précision
Whisper	github.com/openai/whisper	STT offline OpenAI
pynput	pynput.readthedocs.io	Contrôle clavier/souris
pyzbar	github.com/mchehab/zbar	Décodage codes-barres
Google Vision	cloud.google.com/vision	API Cloud Vision
SpeechRecognition	github.com/ Mybigfish/SpeechRecognition	Wrapper STT Python

8.3 Tutoriels Recommandés

- **Real Python** (realpython.com) — Tutoriels de qualité professionnelle :
 - Making an OCR Pipeline in Python
 - Speech Recognition in Python
 - Reading QR Codes with Python
- **Towards Data Science** (towardsdatascience.com) — Articles sur l'OCR avancé et la reconnaissance vocale.

- **TensorFlow Tutorials** (www.tensorflow.org/tutorials) — Guides pour construire des modèles de reconnaissance de caractères.

8.4 Jeux de Données pour Pratiquer

- **MNIST** (yann.lecun.com/exdb/mnist) — 70 000 images de chiffres manuscrit (28x28 pixels).
- **IAM Handwriting Database** — Base de données d'écriture manuscrit en anglais (formulaires scannés).
- **ICDAR Datasets** (icdar2021.irisa.fr) — Jeux de données de compétitions internationales en reconnaissance de texte.
- **LibriSpeech** (www.openslr.org/12) — 960 heures d'audio anglais pour l'entraînement STT.
- **Common Voice** (voice.mozilla.org) — Base de données audio multilingue, incluant le français.

8.5 Aspects Légaux et Éthiques

- **CNIL** — Guide sur la reconnaissance vocale et la biométrie (cnil.fr).
- **RGPD** — Réglementation européenne sur la protection des données : attention lors du traitement de la parole et de l'écriture (données biométriques).
- **Accessibilité** — Les technologies OCR et STT sont cruciales pour l'accessibilité numérique (WCAG 2.1).
- **Biais des modèles** — Les modèles de reconnaissance peuvent avoir des performances différentes selon les accents, les langues ou les styles d'écriture. Il est important de tester sur des données diversifiées.

Conseil

Pour un démarrage rapide, commencez par Tesseract pour l'OCR et Whisper ([small](https://github.com/openai/whisper)) pour la reconnaissance vocale. Ces deux outils sont gratuits, open source et fonctionnent offline. Affinez ensuite avec des modèles plus précis selon vos besoins.